

INF322
2023-2024



INF322 : Bases de données

Le Langage SQL

Avril 2024

Valéry MONTHE

valery.monthe@facsciences-uy1.cm

Bureau R114, Bloc pédagogique 1





1. SGBD
2. Présentation du Langage SQL
3. Langage de définition de données (LDD)
4. Langage de Manipulation de données (LMD)
5. Langage de Contrôle de données (LCD)

- Principaux composants
 - Système de gestion de fichiers
 - Gestionnaire de requêtes
 - Gestionnaire de transactions
- Principales fonctionnalités
 - Contrôle de la redondance d'information
 - Partage des données (car plusieurs utilisateurs en même temps)
 - Gestion des autorisations d'accès
 - Vérification des contraintes d'intégrité
 - Sécurité et reprise sur panne



■ Niveau interne ou physique

- Plus bas niveau
- Indique **comment** (avec quelles structures de données) sont stockées physiquement les données

■ Niveau logique ou conceptuel

- Décrit par un schéma conceptuel ou logique
- Indique quelles sont les données stockées et quelles sont leurs relations, indépendamment de l'implantation physique

■ Niveau externe ou vue

- Propre à chaque utilisateur
- Décrit par un ou plusieurs schémas externes



Structured Query Language (normalisé en 1986)

- SQL2/SQL92 : standard adopté en 1992
- SQL3/SQL99 : extension de SQL2 avec « gestion » d'objets, déclencheurs, etc.
- ...
- SQL2003: auto-incrémentation des clés, colonne calculée, prise en compte de XML, ...
- SQL2008: correction de certains défauts et manques (fonctions, types, curseurs...)



SQL

- **Langage de Définition de Données (DDL ou LDD)** : définir le schéma de la base de données
- **Langage de Manipulation de Données (DML ou LMD)** : interroger et modifier les données de la base
- **Langage de Contrôle d'accès aux Données (DCL ou LCD)** : définir les privilèges d'accès des utilisateurs
- **Langage de Contrôle de Transaction (TCL ou LCT)** : gérer les transactions



Langage de définition de données



<i>Type</i>	<i>Taille (Octets)</i>	<i>Signification</i>
Int	4	Valeur Entière
SmallInt	2	Valeur Entière
TinyInt	1	Valeur Entière
float	4/8	Valeur Décimale
Char (longueur)	Fixe (max 255)	Chaîne de caractères
VarChar (longueur)	Var (max 255)	Chaîne de caractères
Text	Var (max $2^{31}-1$)	Chaîne de caractères
Image	Var (max $2^{31}-1$)	Chaîne binaire

<i>Type</i>	<i>Taille (Octets)</i>	<i>Signification</i>
Bit	1	Valeur Binaire
Binary	Fixe (max 255)	Chaîne binaire
Varbinary	Var (max 255)	Chaîne binaire
Money	8	Valeur en \$
DateTime	2×4 octets	Nb Jours depuis 1/1/1900 + Heure



- La commande **CREATE**
- La commande **ALTER TABLE**
- La commande **DROP**



- Création de la base de données

```
CREATE DATABASE nomDB;
```

Exemple

```
CREATE DATABASE Bibliotheque;
```



■ Création de table

CREATE TABLE nomTable (*col1 type1, col2 type2, ..., coln typen*);

Exemple

Créer la table Stage

```
CREATE TABLE STAGE
  (NumStage INTEGER(3) NOT NULL PRIMARY KEY,
  LibelleStage VARCHAR2(15) NOT NULL,
  Nbjours INTEGER(2),
  TypeStage VARCHAR2(15),
  NumCat INTEGER);
```

Créer la table Stock1(Piece, Fournisseur, NbP)

```
CREATE TABLE Stock1 (
  Piece VARCHAR(20) NOT NULL,
  Fournisseur CHAR(20) NOT NULL,
  NbP INT,
  PRIMARY KEY (Piece, Fournisseur)
);
```



■ Création de d' Index

- Améliorer les temps de réponse des requêtes
- Est automatiquement créé avec la contrainte PRIMARY Key

```
CREATE INDEX nom_index ON nom_table (nom_colonne);
```

```
DROP INDEX nom_index
```



■ Modification d'une table

```
ALTER TABLE table  
{  
  ADD  
  MODIFY (colonne1 type1, colonne2 type2 ..., colonnen typen)  
  DROP
```



■ Modification d'une table

- *Renommer une table*

ALTER TABLE nom_table RENAME TO nouveau_nom;

- *Ajout ou modification de colonne*

ALTER TABLE nom_table {ADD/MODIFY} ([nom_colonne type [contrainte], ...]) ;

- *Renommer une colonne*

ALTER TABLE nom_table RENAME COLUMN ancien_nom TO nouveau_nom ;

- *Supprimer une colonne*

ALTER TABLE nom_table DROP COLUMN nom_colonne ;

- *Ajout d'une contrainte de table*

ALTER TABLE nom_table ADD [CONSTRAINT nom_contrainte]contrainte ;

- *Supprimer des contraintes*

ALTER TABLE nom_table DROP CONSTRAINT nomContrainte ;



■ Modification d'une table : Exemple

- ALTER TABLE STAGIAIRE
ADD NomEntr VARCHAR2(20);
- ALTER TABLE STAGIAIRE
MODIFY NomEntr VARCHAR2(30);
- ALTER TABLE STAGIAIRE
DROP NomEntr;



■ Supprimer une Base de données

- DROP DATABASE nomDB
- Exemple : **DROP DATABASE** bibliotheque;

■ Supprimer une table

- DROP TABLE nomTable
- Exemple : **DROP TABLE** STAGIAIRE;



- Clé primaire
 - **PRIMARY KEY**
- Valeur nulle interdite
 - **NOT NULL**
- Unicité de valeur
 - **UNIQUE**
- Validité de données (restriction de domaine)
 - **CHECK**
- Intégrité référentielle
 - **REFERENCES**



■ Exemple 1 :

```
CREATE TABLE Super_heros(  
    nom_super VARCHAR(20) PRIMARY KEY,                -- UNIQUE NOT NULL  
    nom_civil VARCHAR(20),  
    prenom VARCHAR(20),  
    age INTEGER(2) (CHECK age BETWEEN 1 AND 65));
```

■ Exemple 2:

```
CREATE TABLE Personne(  
    n_ss CHAR(13) PRIMARY KEY,  
    nom VARCHAR(25) NOT NULL,  
    prenom VARCHAR(25) NOT NULL,  
    age INTEGER(3) CHECK (age BETWEEN 18 AND 65),  
    mariage CHAR(13) REFERENCES Personne(n_ss),  
    code_postal INTEGER(5),  
    pays VARCHAR(50),  
    CONSTRAINT ck_Personne UNIQUE (nom, prenom),  
    CONSTRAINT fk_Personne_Pays FOREIGN KEY (code_postal, pays)  
    REFERENCES Adresse (c_p, pays));
```



■ Exemple 3 :

```
CREATE TABLE Enseignant
(
    Enseignant_ID integer,
    Departement_ID integer NOT NULL,
    Nom varchar(25) NOT NULL,
    Prenom varchar(25) NOT NULL,
    Grade varchar(25)
    CONSTRAINT CK_Enseignant_Grade
    CHECK (Grade IN ('Vacataire', 'Moniteur', 'ATER', 'MCF', 'PROF')),
    Telephone varchar(10) DEFAULT NULL,
    Fax varchar(10) DEFAULT NULL,
    Email varchar(100) DEFAULT NULL,
    CONSTRAINT PK_Enseignant PRIMARY KEY (Enseignant_ID),
    CONSTRAINT "FK_Enseignant_Departement_ID"
    FOREIGN KEY (Departement_ID)
    REFERENCES Departement (Departement_ID)
    ON UPDATE RESTRICT ON DELETE RESTRICT
);
```

Contrainte de
domaine

Définition de
clé étrangère



Langage de Manipulation de données



Syntaxe

- *Pour insérer un tuple complètement spécifié :*
INSERT INTO Table VALUES (val1 ,..., valn);
- *Pour insérer un tuple incomplètement spécifié :*
INSERT INTO Table(liste_attributs) VALUES (val1 ,..., valn);
- *On peut insérer un tuple à partir d'une relation ayant le même schéma*
INSERT INTO Table
SELECT *
FROM



☐ Sur les relations

Etudiants (matricule, Nom, Prénom, Age, Ville)

Enseignant (codeProf, Nom, Prénom)

☐ Ajouter l'étudiant Paul PAPA, 21 ans, habitant Douala, ayant pour matricule 20U0215.

```
INSERT INTO Etudiants VALUES ('20U0215', 'PAPA', 'Paul', 21, 'Douala');
```

☐ Ajouter tous les étudiants de Yaoundé dans la liste des enseignants

```
INSERT INTO Enseignant
```

```
SELECT matricule, nom, prenom
```

```
FROM Etudiant
```

```
WHERE ville = 'Yaoundé';
```



Syntaxe

UPDATE Table

SET attribut₁ = expr₁, Attribut_n = expr_n

WHRE condition;

- *Les expressions peuvent être :*
 - ✓ Une constante
 - ✓ Une valeur NULL
 - ✓ Une clause SELECT



❑ Sur la relation

Etudiants (matricule, Nom, Prénom, Age, Ville)

❑ Augmenter l'âge de Paul PAPA (matricule = 20U0215) d'un an.

UPDATE Etudiants

SET age = age + 1

WHERE matricule='20U0215';

❑ Tous les étudiants de Douala on déménagé à Garoua

UPDATE Etudiants

SET ville='Garoua'

WHERE ville = 'Douala';



Syntaxe

DELETE FROM Table
[WHERE condition];

 Exemple : retirer de la liste tous les étudiants de plus de 50 ans.

DELETE FROM Etudiants
WHERE age > 50;



□ Syntaxe : forme générale

```
SELECT [UNIQUE | DISTINCT ] liste_attributs | *  
FROM Table ;
```

Équivalent AR : $\Pi_{\text{liste_attributs}} R(\text{Table})$



☐ Soit la relation

Etudiants (matricule, Nom, Prénom, Age, Ville)

☐ Afficher tous les étudiants

```
SELECT * FROM Etudiants;
```

☐ Noms, prénoms et âges de tous les étudiants

```
SELECT nom, prenom, age
```

```
FROM Etudiants;
```



□ Syntaxe : forme générale

SELECT * FROM table WHERE condition;

Équivalent AR : $\sigma_{condition} R(Table)$

□ La condition peut être formée sur des noms d'attributs ou des constantes avec :

- des opérateurs de comparaison : =, <, >, <> (!=) , <=, >=
- des opérateurs logiques : **AND, OR, NOT**
- des opérateurs : **IN, BETWEEN ... AND, LIKE, EXISTS, IS, ALL, SOME, ANY**
- « _ » remplace un caractère et « % » remplace une chaîne de caractères



□ Syntaxe : forme générale

SELECT <liste de projection>

FROM <liste de tables>

[WHERE <critère de jointure> AND <critère de restriction>]

[GROUP BY <attributs de partitionnement>]

[HAVING <critère de restriction>]

[ORDER BY expression]

[LIMIT count]

□ Restrictions

Les conditions peuvent faire appel aux éléments ci-après :

- Opérateurs logiques : **AND, OR, NOT**
- Comparateurs : **IN, BETWEEN, LIKE** (% , _)
- Comparateurs arithmétiques : **=, <, >, <>, <=, >=, etc.**
- Fonctions sur chaîne : **LENGTH, CONCAT, SUBSTRING**, etc.

□ Possibilité de blocs imbriqués par : **IN, EXISTS, NOT EXISTS, ALL, SOME, ANY**



SQL permet l'imbrication de sous-requêtes au niveau de la clause WHERE.

□ Syntaxe : forme générale

SELECT <liste de projection>

FROM <liste de tables>

WHERE <critère de restriction contenant une requête imbriquée>

□ Restrictions

Les sous requêtes sont utilisées :

- dans des prédicats de comparaison : =, <, >, <>, <=, >=, etc.
- dans des prédicats: **IN, EXISTS, NOT EXISTS, ALL, SOME, ANY**

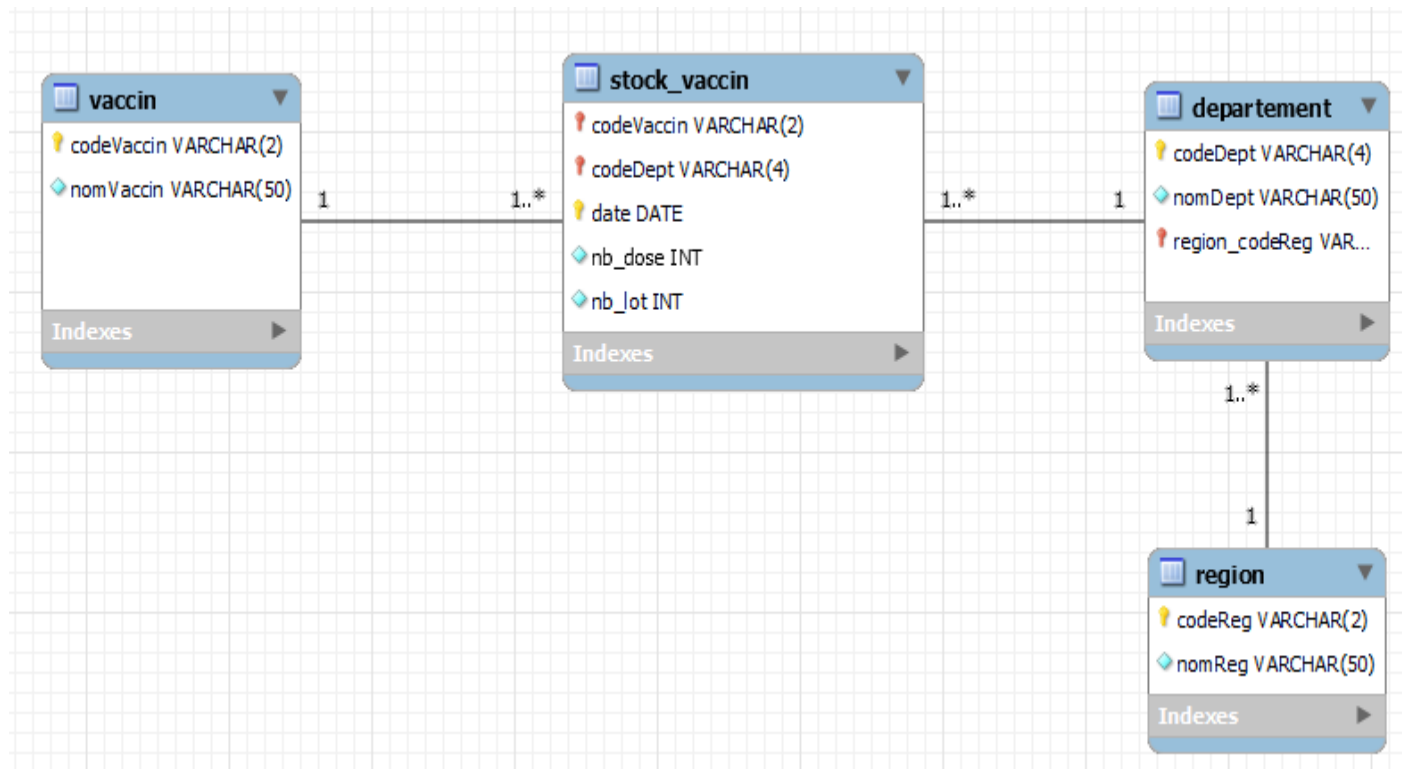


■ Cas d'étude

- On considère une base de données de gestion des stocks de vaccin contre le covid19 en France.

■ Schéma de la base de données

■ Nom de la BD : covid19





- Insertion, modification, suppression
- Sélection / recherche
 - *Requêtes simples*
 1. Liste des départements et le nom de leur région
 2. Tous les départements de la Bretagne
 - *Requêtes imbriquées*
 3. Les départements de la même région que **Paris**
 - *Prédicats du WHERE : LIKE, IN, BETWEEN, NOT, ANY, ALL*
 4. Les départements d'une région autre que celle de **Paris**
 5. Départements d'une région dont le nom contient le mot «**France**»
 - *Agrégation des résultats : COUNT, SUM, AVG, MAX, MIN*
 6. Le nombre de livraisons de chaque type de vaccin
 7. Le nombre de lots livrés par type de vaccin
 8. Le vaccin qui a été le plus utilisé
 9. Le vaccin le moins utilisé



Ouvrages recommandés

- G. Gardarin, Bases de Données-Objet et Relationnelle, Eyrolles. 2001
- C.J. Date. Introduction aux Bases de Données. Vuibert Informatique. 2000
- J. Akoka, I. Comyn-Wattiau. Conception des Bases de Données Relationnelles en Pratique : Concepts, Méthodes et Cas Corrigés. Vuibert Informatique. 2001

Notes de cours et autres références

- Thierry Lecroq, Base de données, Université de Rouen.
- Elsa NEGRE, Bases de données relationnelles – SQL, Lamsade, Paris Dauphine